クラス図の書き方とは。初心者にもわかりやすく解説

by Cacooブログ編集部 2020年5月25日 シェア

Tips (https://cacoo.com/ja/blog/categories/tips-tutorials/)

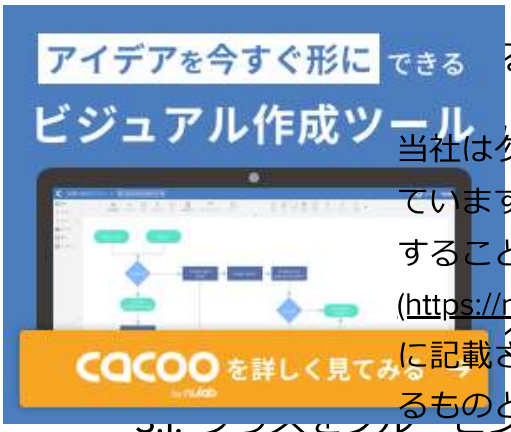
この記事ではUML（統一モデリング言語）の代表的なもののひとつである「クラス図」について、用語の意味から作成方法、分かりやすく書くコツまで解説します。

用語の意味を正しく理解することは作業の効率化に繋がります。ぜひこの記事で理解を深めてくださいね。

目次

1. クラス図とは
    1.1. クラス図のメリット
    1.2. クラス図の使用例

2. クラス図を作成してみよう
    2.1. クラス図の書き方・ルールを解説



(https://cacoo.com/ja/)

るときは「可視性」に注目しよう

する方法とは

コツ

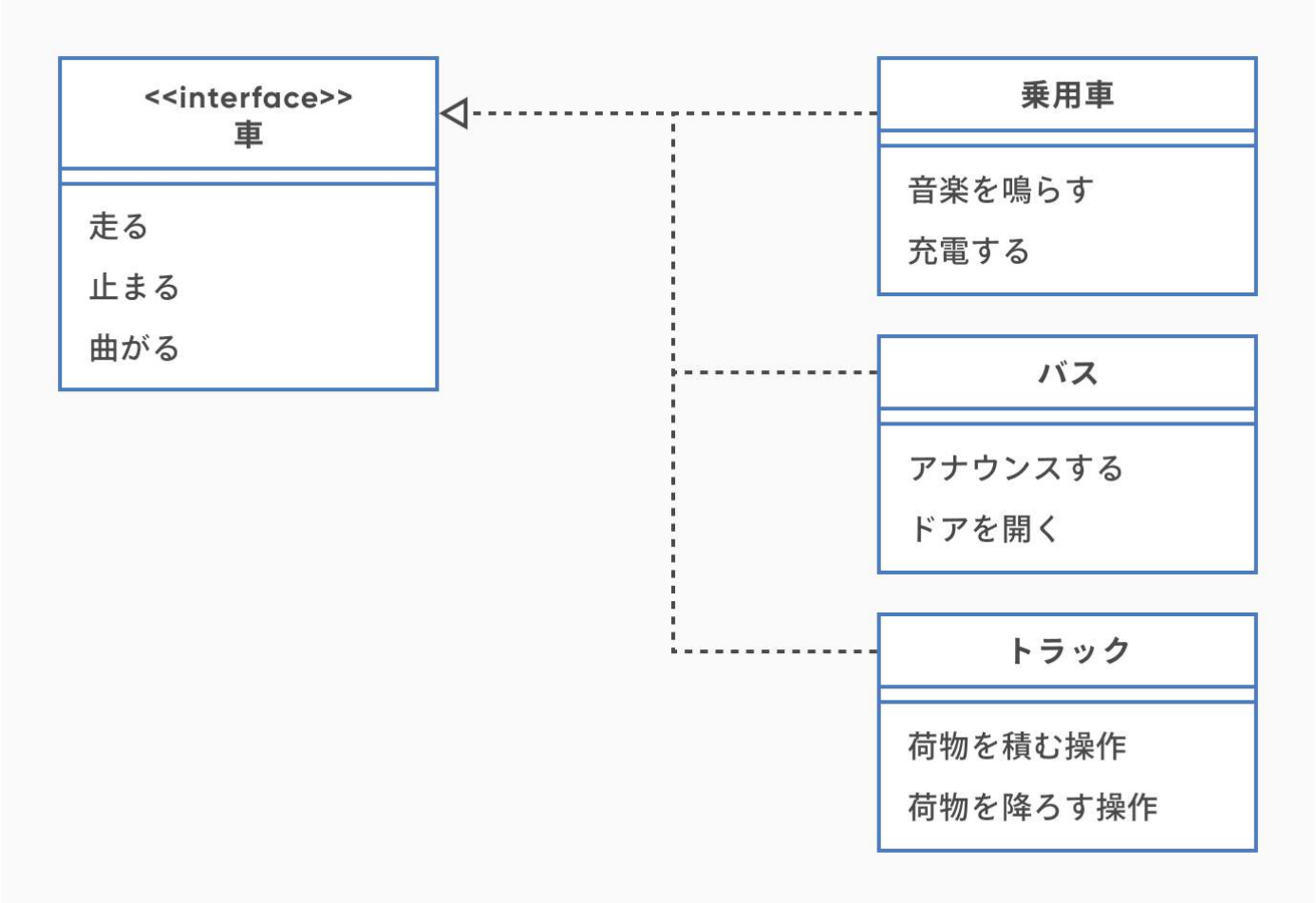
クリックする

当社はクッキーを使ってサービスを提供しています。当社のウェブサイトにアクセスすることにより、クッキーポリシー (https://nulab.com/ja/privacy/cookie-policy/) に記載されているクッキーの使用に同意するものとします。

3.2. 線は上から下、左から右に書く

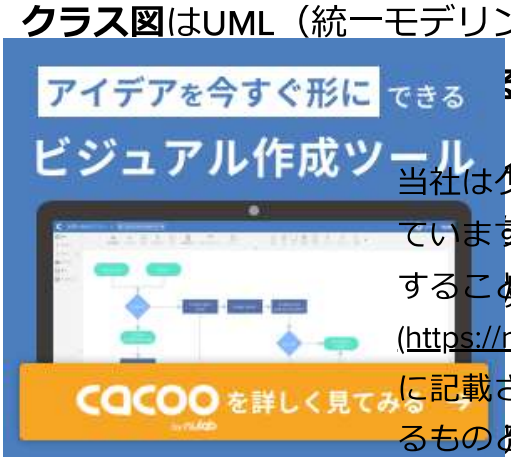
4. クラス図はUMLの基本

クラス図とは



クラス図はUML（統一モデリング言語）のひとつで、特徴は**「システムの静的な構**
るための図」であることです。

当社はクッキーを使ったサービスを提供し
ています。当社のウェブサイトを閲覧
することにより、クッキーの使用に同意す
ることになります。
(https://nulab.com/ja/privacy/cookie-policy/)
に記載されているクッキーの使用に同意す



(https://cacoo.com/ja/)

イントです。たとえばシステム開発の仕様書がすべて文
ち込んで作成するのも大変な労力です。さらに文字ベー
抜け漏れにも気付きにくいです。

クラス図はシステム間の静的構造・関係性を図で表現
るものとされており、汎用性および保守性に優れています。

主に下記の3点がクラス図を作成するメリットとして挙げられます。

1. システム全体をグラフィカルに表現し、概要の把握と共有がしやすい。

2. クラス図を参考に、別のUML図を使用して詳細を掘り下げやすい。

3. システムの実装をせずに、クライアントに対してシステムの全体像を説明できる。

1番については上記でも解説したとおりです。ルールに基づいた記述で、視覚的にシステムの全体像を表現することができるので「何を目的にして作られたシステムなのか」を分かりやすくメンバー間で共有できます。

2番については、詳しくはこの記事では解説しません。「クラス図」はUMLの手法のひとつです。クラス図以外にも「アクティビティ図」「シーケンス図」などさまざまな手法があります。クラス図はUMLの中でも最も基本的な手法です。つまり、クラス図をもとにデータベース設計用のUML図、システムの処理の流れの確認用UML図などが作成できます。

3番については理解しやすいと思います。グラフィカルにシステム全体を表現すれば、エンジニア・非エンジニアに関わらず直感的に理解しやすくなります。クライアントと認識をすりあわせておくことで、後の仕様変更リスクを抑えることが可能です。

クラス図の使用例

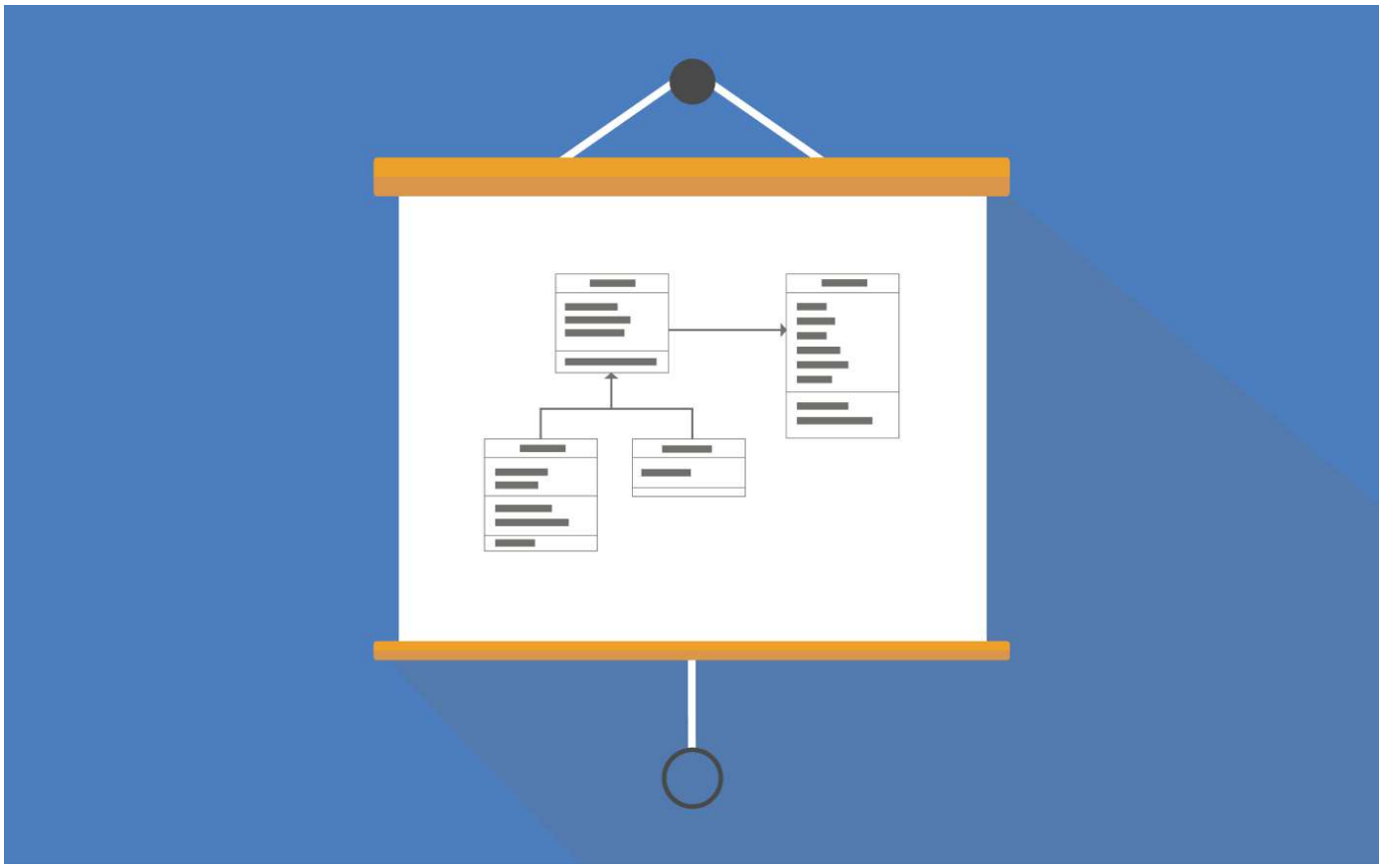
クラス図がイメージしやすいよう、分かりやすい例を挙げてみましょう。

たとえば出退勤の管理システムを単純にクラス図で表現するなら「社員」と「会社」が登場します。そこから掘り下げると、「社員」のシステム（クラス）では日々の出退勤を記録し、「会社」のシステム（クラス）では入社日および退社日について記録

アイデアを今すぐ形にできる
ビジュアル作成ツール
cacooを詳しく見てみる
(https://cacoo.com/ja/)
当社はクッキーを使ってサービスを提供しています。当社のウェブサイトにアクセスすることにより、クッキーポリシー(https://nulab.com/ja/privacy/cookie-policy/)に記載されているクッキーの使用に同意するものとします。

システム（クラス）間のつながりについてイメージできたに、単にイメージを共有することができます。

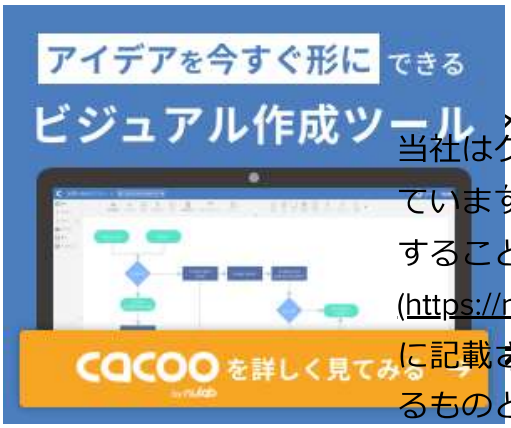
クラス図を作成してみよう



クラス図を作成するには「クラス」と「属性」「操作」に対する理解が欠かせません。先にクラス図の全体像について、概要を説明します。

「クラス」には「属性」と「操作」がついてきます。上記でも使用した、社員管理システムを例にイメージしましょう。
たとえば、「社員」というクラスの中には「社員番号」「名前」「性別」といった属性があります。「操作」はもちろん「出勤」と「退勤」です。

関係するクラスの「会社」ではどうでしょうか。「属性」は会社の名前や住所、操作は、ここでは「入社」と「退社」にしておきましょう。最後に、「社員」クラスと「会社」クラスを線でつなぐことによって、お互いに関係があることが視覚的に理解

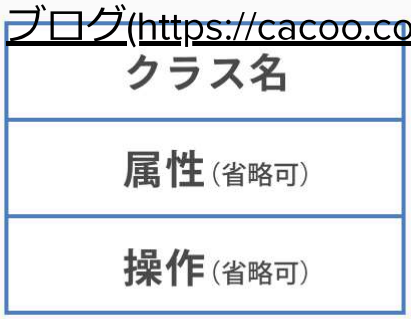


(https://cacoo.com/ja/)

メージを説明しました。以下では、実際にクラス図を1つ一つ、具体的に解説します。

当社はクッキーを使ってサービスを提供しています。当社のウェブサイトにアクセスすることにより、クッキーポリシー (https://nulab.com/ja/privacy/cookie-policy/) に記載されるクッキーの使用に同意するものとします。

を解説



ここからは「**属性**」や「**操作**」について、具体的な書き方を解説しましょう。引き続き、社員の出退勤管理システムを例に解説していきます。

クラスが保有する要素は「属性名」に表される

クラス図を作成するときの第一歩は、登場するクラスをすべて洗い出すことです。出退勤管理システムの例では「社員」と「会社」だけですが、実際のシステム開発では登場するクラスはもっと多いです。

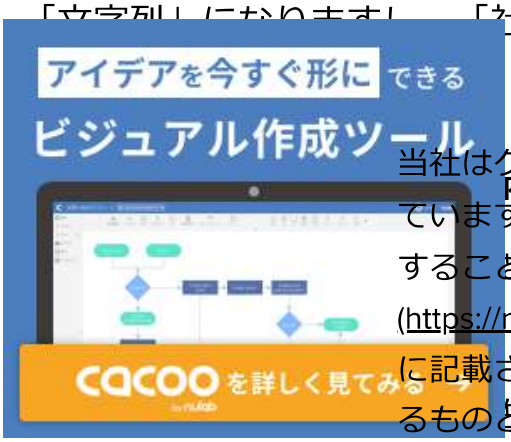
クラスをすべて洗い出せたら、次は「属性」をすべて洗い出して名前を付けましょう。「名前」「住所」「社員番号」「入社日」などが「社員」クラスの　　属性名として考えられます。

ただし、どの要素を実装するかはプロジェクトによって変わります。実際の仕様については、クライアントとよく認識をすり合わせておきましょう。

どのような情報かを表す「型」

属性名の右側には情報の「型」が入ります。属性名が「名前」の場合は「文字列」などといったかんじです。

クラス図においては、実際のプログラミングに使用する型を記述する時もあれば、もうすこし単純に、文字列か数値か程度で表現する時もあります。「住所」であれば「文字列」になります。「社員番号」については型は「数値」であるほうがふさわ



当社はクッキーを使ってサービスを提供しています。当社のウェブサイトにアクセスすることにより、お客様のコンピュータに
(https://nulab.com/ja/privacy/cookie-policy/)
に記載されているクッキーの使用に同意するものとします。

内容を表現しよう

名前のとおり、最初に入れておくべき値（デフォルト値）を記述は「属性名：型＝初期値」と表現されます。

属性の下には「操作」が入ります。操作では、そのクラスで実行可能な内容が記述されています。勤怠管理システムであれば「出社時間」「退社時間」の操作が挙げられます。

最後に、操作プログラムを終了するための「戻り値」を設定しましょう。

以上の内容を記述すると、「操作名（引数：引数の型）：戻り値」と表されます。ちなみに、戻り値を設定しないときは「void」と設定しましょう。

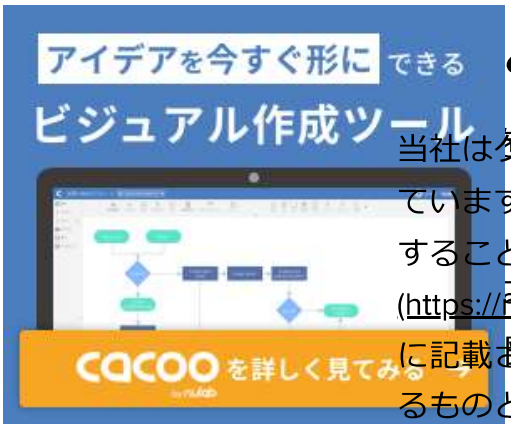
アクセス範囲を見るときは「可視性」に注目しよう

「クラス」「属性」「操作」については理解しました。次は、そのシステムにアクセスできる範囲はどこまでか設定しましょう。システムにアクセスできる範囲を表現するものを「可視性」と言います。

勤怠管理システムの例でいうと、出退勤時間をチェックするには「社員」クラスからしか確認できないのか、もしくは「会社」クラスからでも確認できるか、というイメージです。

クラス図で可視性を表現する場合、要素名や操作の左側に記号で可視性が記入されます。すべて記号で表現されています。可視性は4種類のみなので、比較的覚えやすいかと思います。

「+」・・・すべてのクラスからアクセス可能
「-」・・・自クラスのみアクセス可能
「#」・・・自クラスおよび継承されているクラスからアクセス可能
「~」・・・同一パッケージ内のクラスからアクセス可能

×


(https://cacoo.com/ja/)
とは
を例に、可能な限りシンプルにクラス図の使い方を説明します。
当社はクッキーを使ったサービスを提供しています。当社のウェブサイトにアクセスすることにより、クッキーポリシー (https://nulab.com/ja/privacy/cookie-policy/) に記載されているクッキーの使用に同意するものとします。

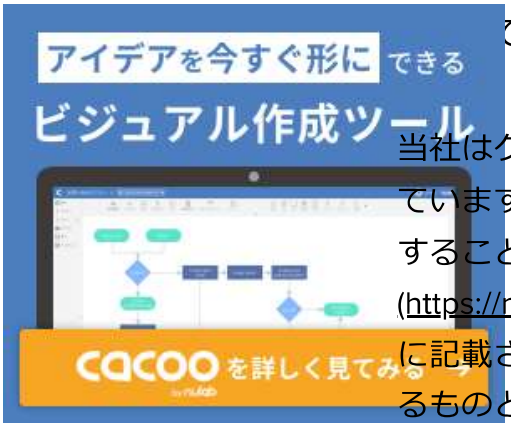
「関連」とは文字通り、クラス間の関係性を記述しています。勤怠管理システムの例だと、「社員」と「会社」クラス間には雇用関係がありますよね。そのときはクラス間を線で結び、線の上に関連名（この場合は「雇用」「所属」など）を記述します。



「多重度」とは、「Aクラスから見た場合のBクラスが存在しうる数」です。少しわかりにくいですよね。勤怠管理システムの例で考えてみましょう。

「会社」クラスから見ると「社員」クラスは（従業員が1人以上存在する限り）複数存在するはずです。逆もしかりで、「社員」クラスからみて「会社」クラスは1つしか存在しません。この場合、クラス間をつなぐ線の端に、多重度を示す記号を表記します。

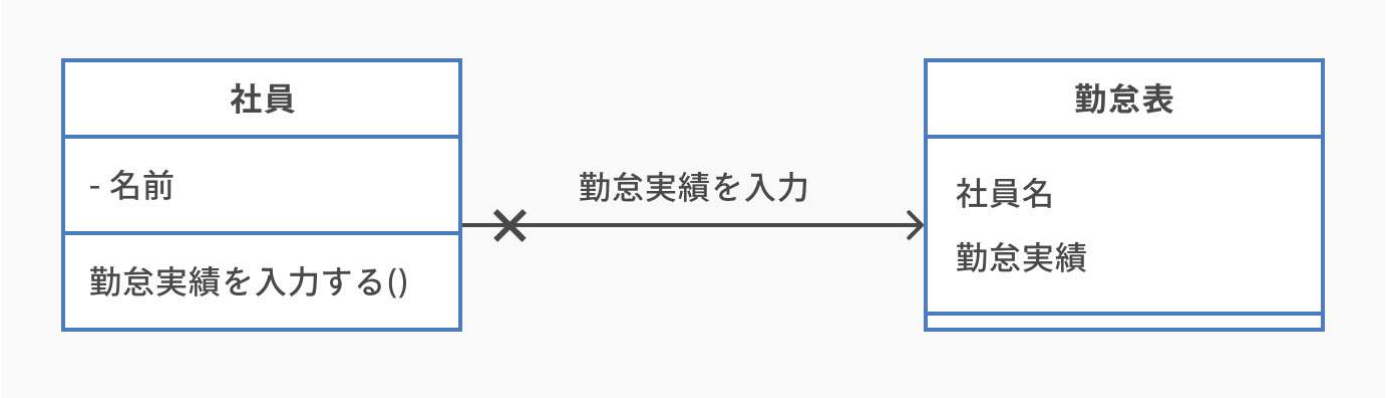


×
です。


(https://cacoo.com/ja/)
当社はクッキーを使ってサービスを提供しています。当社のウェブサイトにアクセスすることにより、クッキーポリシー(https://nulab.com/ja/privacy/cookie-policy/)に記載されているクッキーの使用に同意するものとします。

「**誘導可能性**」は、クラス同士のやりとりの方向性を表現します。勤怠管理システムの例だと、「社員」クラスから「会社」クラスの操作ができることはありえません。逆に、「会社」クラスから「社員」クラスへのアクセスは考えられます。

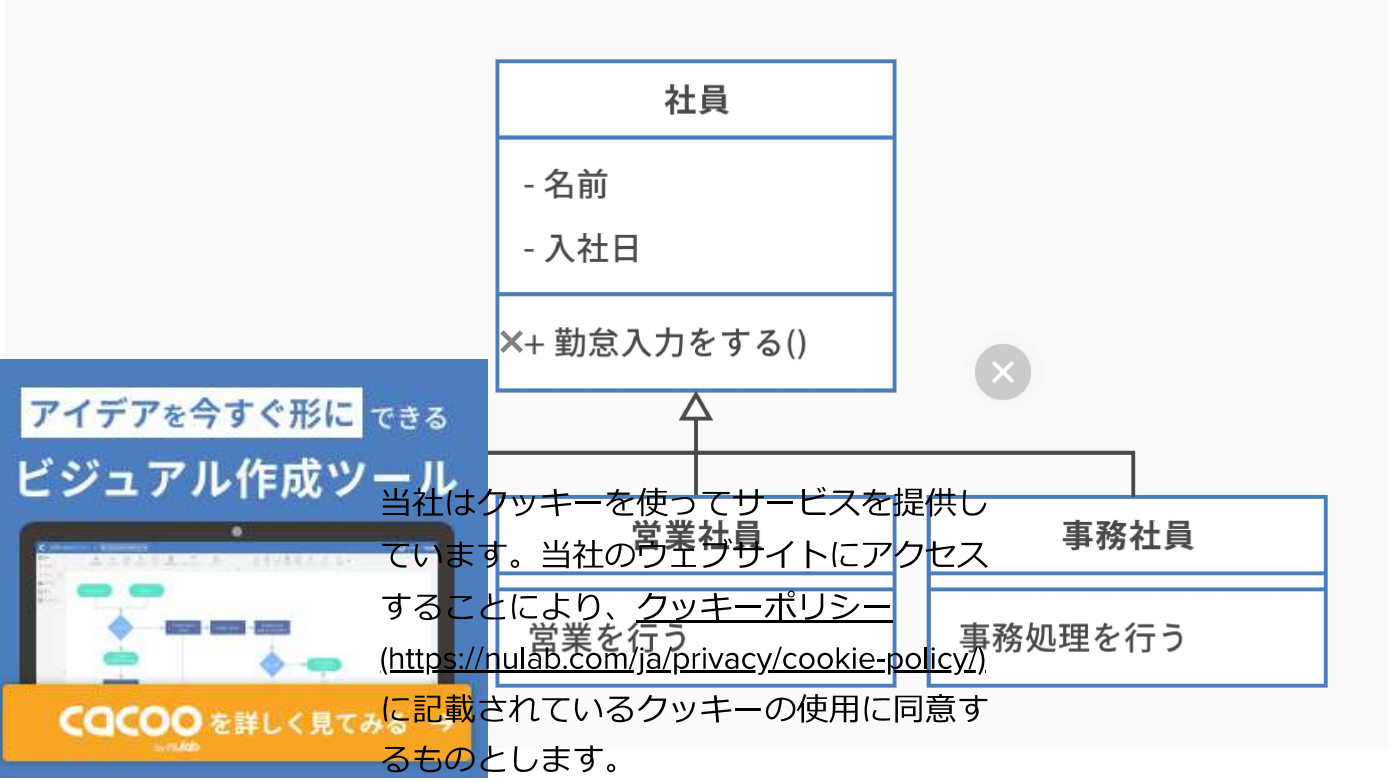
この場合、クラス間の関係を表す線は矢印とバツで表現します。



クラスの型を継承する「汎化」「特化」「実現」とは

あるクラスAの型を継承してより具体化したクラスBを作成した場合、この2クラスの関係性は具体的なクラスBと抽象的なクラスAという意味で「**凡化**」関係と言えます。

たとえば「営業社員」「事務社員」「技術社員」などのクラスを、より抽象的な「社員」クラスへと凡化することが可能です。また、逆に「社員」クラスからより具体的な「営業社員」「事務社員」「技術社員」などのクラスに具体化することを「特化」と言います。



アイデアを今すぐ形にできる
ビジュアル作成ツール
cacoo を詳しく見てみる
(https://cacoo.com/ja/)
当社はクッキーを使ってサービスを提供しています。当社のウェブサイトにアクセスすることにより、クッキーポリシー(https://nulab.com/ja/privacy/cookie-policy/)に記載されているクッキーの使用に同意するものとします。

下記で詳述する「インターフェース」ともつながりますが、共通操作を定義したクラスと個別動作を定義したクラス間の関係を「実現」と言います。詳しくは下記「インターフェースとは？」を参考にしてください。

クラスの「集約」と「コンポジット」とは

クラス同士の関係が「全体と部分」であるときに「**集約**」として表現されます。「会社」クラスという全体があり、「社員」クラスという部分があるイメージです。



「**コンポジット**」は集約の一種で、「全体」クラスが「個別」クラスの生成や削除の権利があるときに使用されます。「会社」クラスが「社員」クラスを自由に生成・削除可能な場合はコンポジットであると言えます。



クラス図で使われる「依存」は弱い関係性を表す？

アイデアを今すぐ形にできる
ビジュアル作成ツール
cacoo を詳しく見てみる
当社はクッキーを使ったサービスを提供しています。当社のウェブサイトにアクセスすることにより、クッキーポリシー (https://nulab.com/ja/privacy/cookie-policy/) に記載されているクッキーの使用に同意するものとします。

言葉があります。依存というと密接な関係をイメージしちょっと違う意味なので注意しましょう。

ラス間の弱い関係性がある」場合に使われます。たとえ別に「部署」クラスがあると仮定しましょう。ラスを生成・削除できますが、普段は「部署」クラスをるものとします。「会社」「社員」クラスはつながりが弱いです。この場合、会社と社員クラスは「依存」であると言えます。

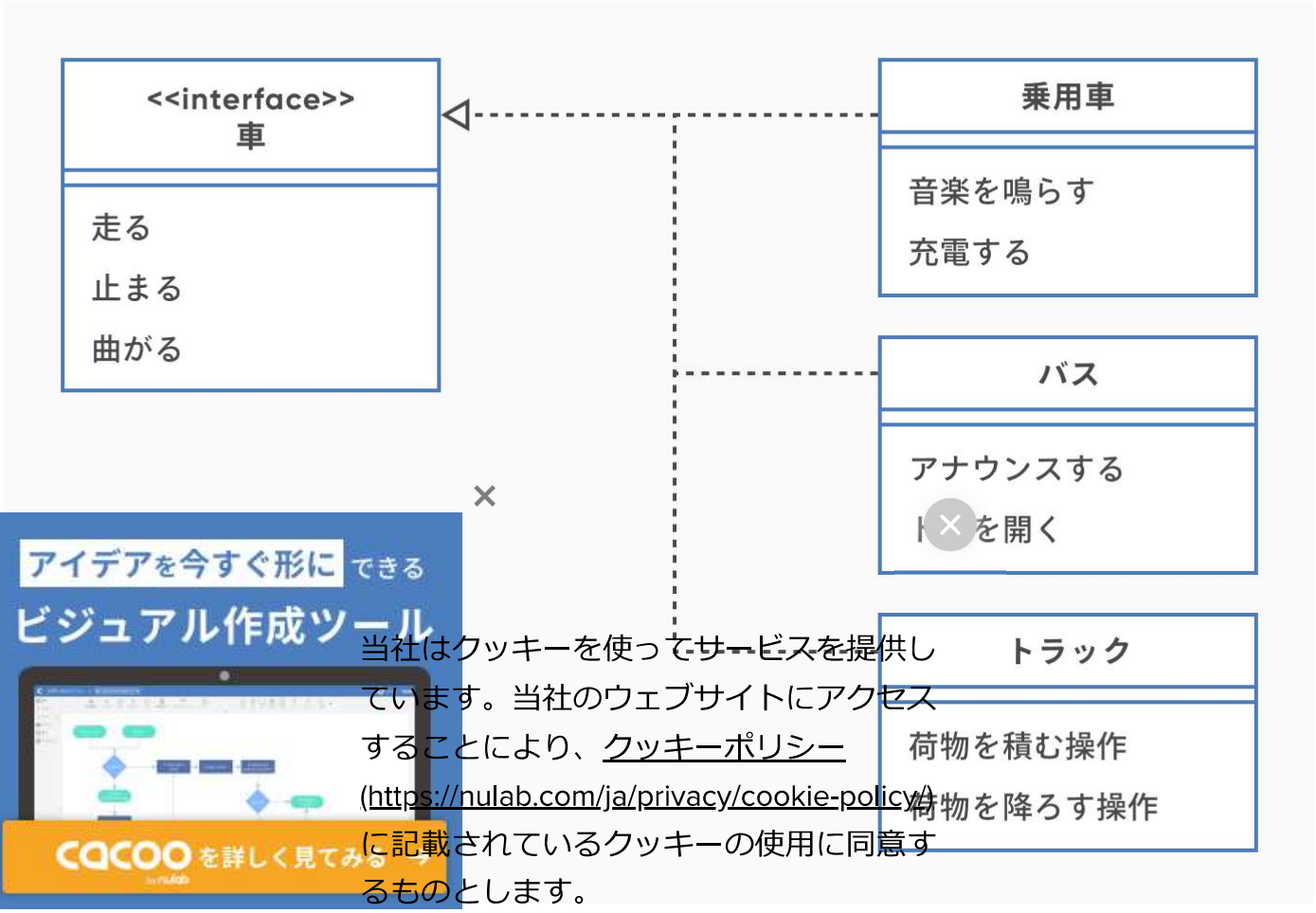


クラス間で共通している「インターフェース」とは？

インターフェースとは「共通した機能のみを定義したクラス」のことを指します。ここでは「車を操作するシステム」を例にイメージしましょう。

自動車・バス・トラックなど、車は種類を問わず「走る・止まる・曲がる」という操作が必須です。この必須動作を記述したクラスを「インターフェース」と言います。

なお、インターフェースと関係のあるクラスを「サブクラス」と言います。車のインターフェースであれば、サブシステムは「自動車」「バス」「トラック」が挙げられます。個別要素として、バスであれば「アナウンスをする」「ドアを開く」、トラックであれば「荷物の積み降ろし操作」などが挙げられます。



アイデアを今すぐ形に できる
ビジュアル作成ツール
cacoo を詳しく見てみる
(https://cacoo.com/ja/)

当社はクッキーを使ってサービスを提供しています。当社のウェブサイトにアクセスすることにより、クッキーポリシー(https://nulab.com/ja/privacy/cookie-policy/)に記載されているクッキーの使用に同意するものとします。

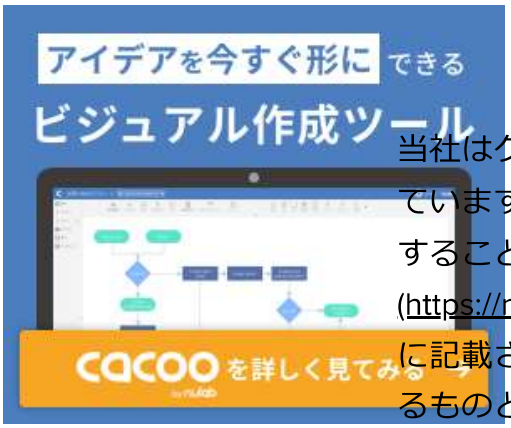
クラス図の細かい使い方について、用語の理解はできたでしょうか。しかし、クラス図を作るためには用語の理解だけでなく作成まで対応しないといけません。

矢印の意味は上記ですべて説明しました。以下に説明した矢印をまとめています。矢印の種類と意味を一致させられるようにしましょう。

線形	意味
	関連
	集約
	コンポジット
	依存
	汎化
	実現
	誘導可能性

矢印の種類まとめ

見やすいクラス図を書くコツ



(https://cacoo.com/ja/)

当社はクッキーを使ってサービスを提供しています。当社のウェブサイトにアクセスすることにより、クッキーポリシー(https://nulab.com/ja/privacy/cookie-policy/)に記載されているクッキーの使用に同意するものとします。



ここまで読んで、実際にクラス図を作成するイメージは湧きましたでしょうか。読んだだけでは難しいかもしれませんが、クラス図の利用は慣れです。日々の業務を頑張りましょう。

最後に、クラス図を上手に書くちょっとしたポイントをご紹介します。コツを意識するだけで見やすいクラス図を作成することができますので、ぜひ試してみましょう。

クラスをグループピングする

クラス図を洗い出したとき、できるだけ機能や役割に関係があるクラスをちかくにまとめておきましょう。これを「グループピング」と言います。

アイデアを今すぐ形にできる
ビジュアル作成ツール
cacoo を詳しく見てみる
(https://cacoo.com/ja/)

ラスをまとめて四角形でくくりましょう。そのうえで、
ておくと分かりやすいです。
の つながりを示す線が 長に
なり、見にくいクラス図
の洗い出しをするときに気をつけるか、洗い出し後に
ましょう。

当社はクッキーを使ってサービスを提供しています。当社のウェブサイトをご利用になることにより、クッキーポリシー (https://nulab.com/ja/privacy/cookie-policy/) に記載されているクッキーの使用に同意するものとします。

クラス図は静的なシステムのつながりを表現した図ですが、時系列もできるだけ分かりやすくすると読みやすくなります。

一般的にはプログラム実行順に、はじめに動くほうが上もしくは左に記載しましょう。上から下、左から右にシステム処理のフローも記載すると良しです。

クラス図はUMLの基本

簡単そうで意外とルールが多いクラス図について解説しました。

UMLにはほかにもさまざまな手法があります。しかし、あれこれと手を出すのではなく、まずはクラス図について理解を深めるのをおすすめします。なぜなら、ほかのUML図でも「クラス」の概念を理解していないと正しく理解するのが難しいからです。

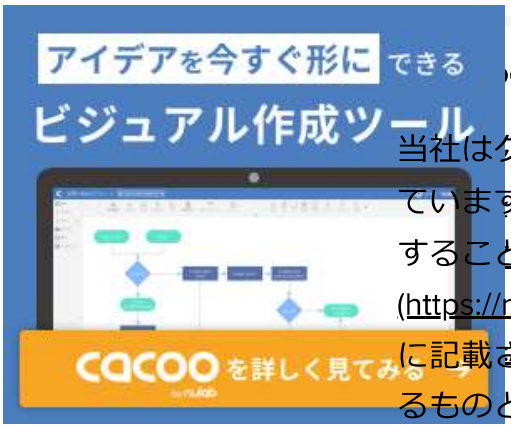
繰り返しになりますが、クラス図は実際に作ってみる・読んでみることで上手に活用できるようになります。実践の中で慣れていきましょう。



記事の監修　砂川 祐樹

Backlog開発チームで開発を担当する傍ら、プロジェクト管理について噛み砕いて解説する入門サイト、サル先生のプロジェクト管理入門 (https://backlog.com/ja/project-management-guide/) や サル先生のバグ管理入門 (https://backlog.com/ja/bug-tracking-guide/)の制作に携わった他、プロジェクト管理を楽しく学べるボードゲーム「プロジェクトテーマパーク」 (https://backlog.com/ja/blog/making-of-project-themepark/)を制作。プロジェクト管理を身近なものとして広めるヌーラボの活動に関わっている。

×



(https://cacoo.com/ja/)

o編集部

当社はクッキーを使ってサービスを提供しています。当社のウェブサイトにアクセスすることにより、クッキーの使用に同意するものとします。

ーズに関する情報を発信しています。お役に立てた情報があればお願いします。

ームクルセイン資料まで作れる | Cacoo(カクー) (https://cacoo.com/ja/)

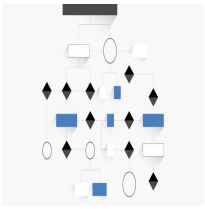
(https://nulab.com/ja/privacy/cookie-policy/)

に記載されている

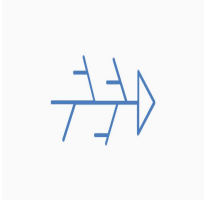
関連する記事



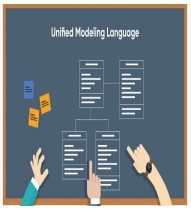
ソコストのイラストを使って、Cacooでプレゼンテーション資料を作ろう！ (https://cacoo.com/ja/blog/soco-st-cacoo-presentation/)



DFD（データフロー図）ってなに？DFDの概要と書き方をあわせて紹介 (https://cacoo.com/ja/blog/what-is-dfd/)



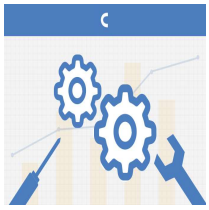
QC7つ道具の「特性要因図」とは？書き方や使用用途について解説 (https://cacoo.com/ja/blog/what-is-fishbone-chart/)



UMLとは？書き方とクラス図・シーケンス図など10種の図を解説 (https://cacoo.com/ja/blog/what-is-uml/)



Visioの代替はCacooがおすすめ！作図ツールを選ぶときの3つのポイント (https://cacoo.com/ja/blog/cacoo-visio-alternative-collaborators/)



【Cacoo塾6限目】見やすいグラフを簡単に作成しよう！スタイル調整の仕方と機能アップデート (https://cacoo.com/ja/blog/how-change-cacoo-dynamic-chart-styling/)



【Cacoo塾3限目】コメントのアップデート機能を使いこなす！改善点と特殊な記法を紹介 (https://cacoo.com/ja/blog/comment-feature-update/)



クリック表示・スナップ (https://cacoo.com/ja/blog/how-to-set-snap-grid-and-non-connected-line/)



をCacooで記録してチーム (https://cacoo.com/ja/blog/cacoo-version-history-how-to-use/)

アイデアを今すぐ形にできる
ビジュアル作成ツール
cacoo を詳しく見てみる
(https://cacoo.com/ja/)

当社はクッキーを使ってサービスを提供しています。当社のウェブサイトにアクセスすることにより、クッキーポリシー (https://nulab.com/ja/privacy/cookie-policy/) に記載されているクッキーの使用に同意するものとします。

オンライン コラボレーション作図ツール

世界中のユーザーがなぜ Cacoo を選んでいるのかご自身の目でお確かめください

無料トライアル (https://cacoo.com/ja/pricing)

サービス

特長 (https://cacoo.com/ja/features)

価格 (https://cacoo.com/ja/pricing)

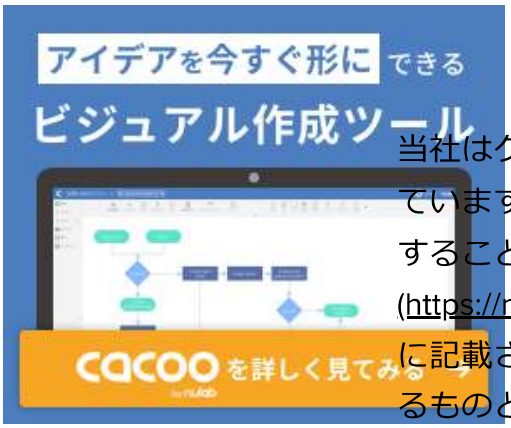
導入事例 (https://cacoo.com/ja/customers)

外部サービス連携 (https://cacoo.com/ja/integrations)

最新情報 (https://cacoo.com/ja/blog/categories/product-updates)

セキュリティ (https://cacoo.com/ja/security)

プラン



当社はクッキーを使ってサービスを提供しています。当社のウェブサイトにアクセスすることにより、クッキーポリシー (https://nulab.com/ja/privacy/cookie-policy/) に記載されているクッキーの使用に同意するものとします。

(https://cacoo.com/ja/)

(https://www.facebook.com/Cacoo.com/) (https://twitter.com/Cacoo) (https://www.linkedin.com/showcase/10531082/)

© 2022 Nulab, Inc. All rights reserved.